DS-GA 1003 Machine Learning: Homework 3

Due 11.59 p.m. EST, April 25, 2024 on Gradescope

(Jiasheng Ni)
(collaborators if any)

We encourage LaTeX-typeset submissions but will accept quality scans of hand-written pages.

1 Variational Inference and Monte Carlo Gradients

In this question, we will review the details of variational inference (VI), in particular, we will implement the gradient estimators that make VI tractable.

We consider the latent variable model $p(\mathbf{z}, \mathbf{x}) = \prod_{i=1}^{N} p(\mathbf{x}_i|\mathbf{z}_i)p(\mathbf{z}_i)$ where $\mathbf{x}_i, \mathbf{z}_i \in \mathbb{R}^D$. Recall that in VI, we find an approximation $q_\lambda(\mathbf{z})$ to $p(\mathbf{z}|\mathbf{x})$.

(A) Let $V_1(\lambda)$ be the set of variational approximations $\{q_\lambda : q_\lambda(\mathbf{z}) = \prod_{i=1}^{N} q(\mathbf{z}_i; \lambda_i)\}$ where λ_i are parameters learned for each datapoint $\mathbf{x}_i$. Now consider $f_\lambda(\mathbf{x})$ as a deep neural network with *fixed* architecture where λ parametrizes the network. Let $V_2(\lambda) = \{q_\lambda : q_\lambda(\mathbf{z}) = \prod_{i}^{N} q(\mathbf{z}_i; f_\lambda(\mathbf{x}_i))\}$. Which of the two families (V_1 or V_2) is more expressive, i.e. approximates a larger set of distributions? **Prove** your answer.

Will your answer change if we let f_λ represent *variable* architecture, e.g. if λ parametrizes the set of multi-layered perceptrons of all sizes? Why or why not?

Solution. (a) We have to show that there is some q_λ in the $V_2(\lambda)$ that is not in $V_1(\lambda)$. Since for $V_1(\lambda)$ case, each $\mathbf{x}_i$ is parametrized by its unique distribution parameters and are assumed to be independent with each other. In $V_2(\lambda)$, the neural network can basically represent any distributions with greater accuary. For example, if we have N data points $\mathbf{x}_1, \cdots, \mathbf{x}_N$ where $q(\mathbf{z}_i, \lambda_i) \sim \mathcal{N}(\mathbf{z}_i; \mu_i, \sigma_i^2)$. suppose the ground true relationship is $z = sin(x) + \epsilon$(where ϵ is some Gaussian noise), the λ_i's in $V_1(\lambda)$ are learned from some MLE/MAP process assuming that z_i and x_i follows a GLM model. In this case, the learned distribution would be just a rough estimate of the true $q(z_i; \lambda_i)$ since we cannot learn gaussian noise. But in $V_2(\lambda)$, the neural network can represent $z = sin(x) + \epsilon$ precisely. So we can easily find a distribution that is in $V_2(\lambda)$ but not in $V_1(\lambda)$ and the proof is finished.

(b) If we let f_λ represent variable architecture, the answer won't change because we can represent even larger set of distribution with variable model architecture.

□

(B) For variational inference to work, we need to compute unbiased estimates of the gradient of the ELBO. In class, we learnt two such estimators: score function (REINFORCE) and pathwise (reparametrization) gradients. Let us see this in practice for a simpler inference problem.

Consider the dataset of $N = 100$ one-dimensional data points $\{x_i\}_{i=1}^{N}$ in `data.csv`. Suppose we want to minimize the following expectation with respect to a parameter μ:

$$\min_{\mu} \mathop{\mathbb{E}}_{z\sim\mathcal{N}(\mu,1)} \left[\sum_{i=1}^{N}(x_i - z)^2\right] \tag{1}$$

(i) Write down the score function gradient for this problem. Using a suitable reparametrization, write down the reparameterization gradient for this problem.

Solution. i. **Score Function Estimator**:

$$\begin{aligned}
\text{Denote } g(z;\mu) &= \sum_{i=1}^{N}(x_i - z)^2 \\
\nabla_\mu ELBO &= \mathbb{E}_{z\sim N(\mu,1)}\left[g(z;\mu,1)\cdot\nabla_\mu \log\frac{1}{\sqrt{2\pi}}\exp\left\{-\frac{1}{2}(z-\mu)^2\right\}\right] \\
&= \mathbb{E}_{z\sim N(\mu,1)}\left[\sum_{i=1}^{N}(x_i - z)^2\left\{\nabla_\mu - \frac{1}{2}(z-\mu)^2 + \nabla_\mu C\right\}\right] \\
&= \mathbb{E}_{z\sim N(\mu,1)}\left[\sum_{i=1}^{N}(x_i - z)^2(z-\mu)\right] \\
&= -2\mathbb{E}_{z\sim N(\mu,1)}[z(z-\mu)]\sum_{i=1}^{N}x_i + \mathbb{E}_{z\sim N(\mu,1)}\left[Nz^2(z-\mu)\right) \\
&= -2\left\{1+\mu^2-\mu^2\right\}\sum_{i=1}^{N}x_i + N\left\{3\mu+\mu^3-\mu\left(1+\mu^2\right)\right\} \\
&= -2\sum_{i=1}^{N}x_i + 2N\mu
\end{aligned}$$

where C is a constant and we use the fact that $\mathbb{E}[z^3] = 3\mu\sigma^2 + \mu^3$ by MGF.

ii. **Pairwise Estimator**: We could pick $z = \mu + \epsilon$ where $\epsilon \sim \mathcal{N}(0,1)$ so that we have:

$$\begin{aligned}
&\nabla_\mu \mathbb{E}_{\epsilon\sim\mathcal{N}(0,1)}\left[\sum_{i=1}^{N}(x_i - (\mu+\epsilon))^2\right] = \mathbb{E}_{\epsilon\sim\mathcal{N}(0,1)}\left[\nabla_\mu\sum_{i=1}^{N}(x_i-(\mu+\epsilon))^2\right] \\
&= \mathbb{E}_{\epsilon\sim\mathcal{N}(0,1)}\left[-2\sum_{i=1}^{N}(x_i-(\mu+\epsilon))\right] \\
&= -2\sum_{i=1}^{N}(x_i-\mu)
\end{aligned}$$

We see in this particular case, the two gradients are the same.
But the gradient descent process is different due to different distribution to sample from. □

(ii) Using PyTorch and for each of these two gradient estimators, perform gradient descent using $M = \{1, 10, 100, 1000\}$ gradient samples for $T = 10$ trials. Plot the mean and variance of the final estimate for μ for each value of M across the T trials.

You should have two graphs, one for each gradient estimator. Each of the graph should contain two plots, one for the means and one for the variances. The x-axis should be M, hence each of these plots will have four points.

Solution. The plot is provided below:

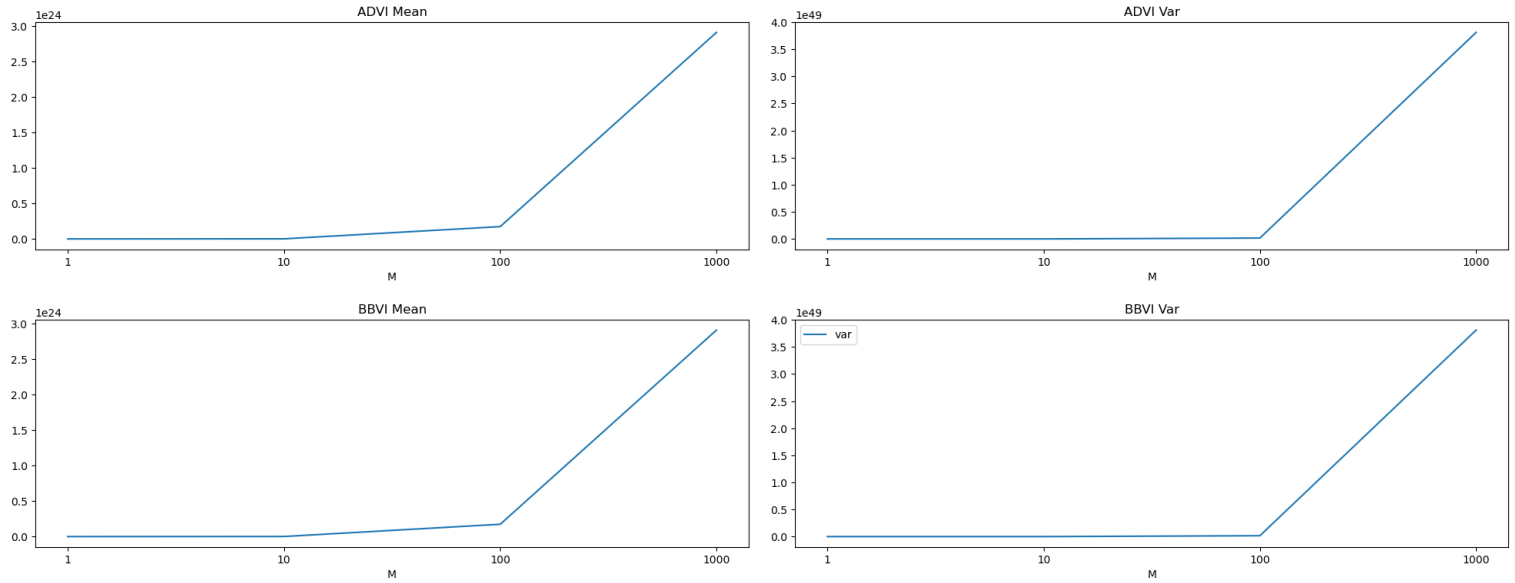


Figure 1: Output Plot

□

(C) What conditions do you require on $p(z)$ and $f(z)$ ($f(z) = \sum_{i=1}^{N}(x_i - z)^2$ in this case) for each of the two gradient estimators to be valid? Do these apply to both continuous and discrete distributions $p(z)$?

Solution. (a) **Score Function Estimator**: p(z) needs to be differentiable almost everywhere(but not strictly enforced) since it relies on the derivative of the log probablity(defined even when z is discrete), and f(z) can be either differentiable or at least measurable with respect to z.

(b) **Pairwise Estimator**: Both $p(z)$ and $f(z)$ needs to be differentiable w.r.t z.

In discrete case, these still applies, it's just that for score function estimator, the variance of the gradient would be higher due to lack of data points. □

2 Normalizing Flows

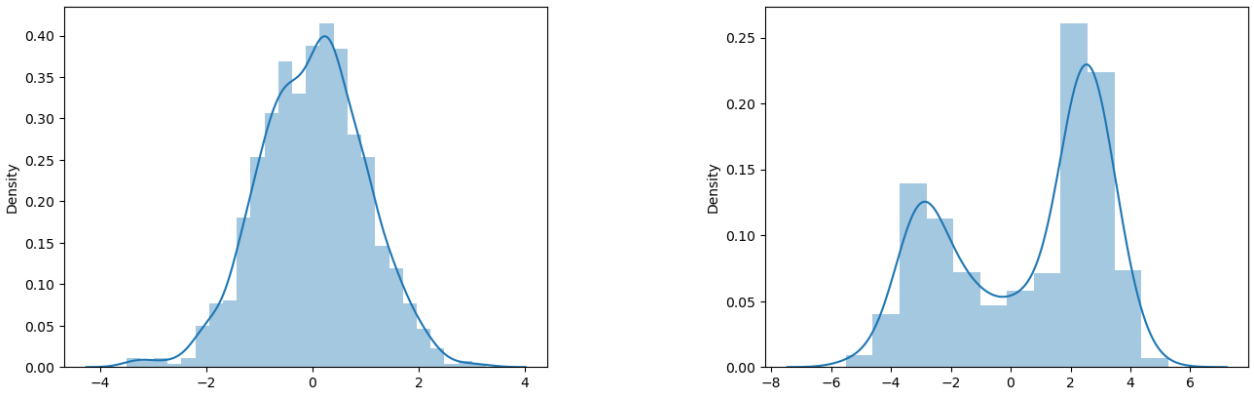
In this question, we will review how we can use invertible transformations and the change-of-variables formula to turn simple distributions into complex ones. Such transformations are known as *normalizing flows*. One reason that flows are useful is that they can map unimodal distributions into multimodal ones, while still allowing for a tractable density.

(A) Let $z_0 \sim \mathcal{N}(0, 1)$. Produce a density plot of z_0 using $N = 1000$ samples.

Now look up "Planar Flow" (Equation 4) of The Expressive Power of a Class of Normalizing Flow Models. Denote this flow as f. Choose an invertible non-linearity h and find values of w, b, u such that $f(z_0)$ is a multimodal distribution. Plot the density plot of $f(z_0)$ using the same $N = 1000$ samples as above.

Note that $d = 1$ in this question. Also, it is fine to choose a h that is only invertible on its output range, e.g. the sigmoid function on $(0, 1)$.

Solution. Here we choose u = 2, w = 3, b = 1, $h(\cdot) = tanh(\cdot)$ to obtain the following graph:



((a)) Original ((b)) After Flow

Figure 2: Original and transformed

□

(B) Use the change-of-variables formula and write down an explicit expression for the density of $f(z_0)$. This depends on your choice of h.

Solution. We choose *tanh* function in part(A). We denote the pdf for $x = f(z_0) \in (-1, 1)$ by $p_f(x)$. Then by the change of variable formula we have:

$$\begin{aligned} p_f(x) &= p_z\left(f^{-1}(x)\right) \cdot \left| \frac{d}{dx}\left(f^{-1}(x)\right) \right| \\ &= p_z\left(f^{-1}(x)\right) \cdot \frac{1}{\left|\frac{d}{dx} f(z)\right|} \\ &= \frac{p_z(f^{-1}(x))}{|1 + u \cdot (1 - tanh^2(w \cdot f^{-1}(x) + b)) \cdot w|} \end{aligned}$$

□

(C) Let's generalize this to D-dimensional variables and to a sequence of invertible functions f (not necessarily planar flows). We can sample $\mathbf{z}^{(0)} \sim \mathcal{N}(\mathbf{0}, I)$ and then transform that sample iteratively using a sequence of invertible functions, $f_1, \ldots, f_K$, to finally obtain a sample $\mathbf{z}^{(K)}$ where

$$\mathbf{z}^{(K)} = f_K \circ \cdots \circ f_2 \circ f_1(\mathbf{z}^{(0)}).$$

Using the change-of-variables formula, write down a formula for the log-density of $\mathbf{z}^{(K)}$ using the functions $\{f_k\}_{k=1}^K$, their inverses and Jacobians, as well as the log-density of $\mathbf{z}^{(0)}$.

Solution. The formula is given by

$$\log p_{z(K)}(x) = \log p_{z(0)}\left(f_1^{-1} \circ \cdots \circ f_K^{-1}(x)\right) + \sum_{k=1}^{K} \log \left| \det \left(J_{f_k^{-1}} \right) \right|$$

□

(D) Let us consider how we can improve variational inference using flows.

(i) How can we define an approximation $q_\lambda(\mathbf{z})$ to $p(\mathbf{z}|\mathbf{x})$ using flows?

Solution. We can define a simple prior distribution for z and propagate it through the flow model(a sequence of flow layer actually) which is parametrized by λ to approximate the $p(z|x)$, which is a complex distribution. □

(ii) How do we train the model, i.e. optimize λ to yield a good approximation to $p(\mathbf{z}|\mathbf{x})$?

Solution. We could define a loss function called Variational Loss, which is computed by ELBO to measure the divergence between the learned distribution $q_\lambda(z)$ and the true target distribution $p(z|x)$. We can then use backpropagation algorithm to update the parameters λ in the model. □

(iii) In what way is this more flexible than using a Gaussian approximation for $q_\lambda(\mathbf{z})$?

Solution. Because flows can transform a simple base distribution into a more complex one that can capture multimodal characteristics and correlations within the data, using Gaussian approximation cannot capture the complex multi-modality of the target distribtion. Even if we use mixture of Gaussian to approximate, we cannot model the multi-modal distribution with spiky density but flow model can do this. □

3 Diffusion Models and Denoising Score Matching

In class, we learned about the ELBO for diffusions. Part of this ELBO included a term that "matched the score," which in a simplified form yields a loss called denoising score matching:

$$\mathcal{L}_{\text{dsm}}(\theta) = \int_{\epsilon}^{1} E_{q(z_0,z_t)}(s_\theta(z_t,t) - \nabla_{z_t} \log q(z_t \mid z_0))^2 dt$$

(A) In terms of q, what is the optimal $s_\theta(z_t, t)$?

Solution. The only term that contains θ is $s_\theta(z_t,t)$ so we can simply take the gradient of $\mathcal{L}_{\text{dsm}}(\theta)$ w.r.t θ and set it to 0 to get

$$2 \cdot \int_{\epsilon}^{1} E_{q(z_0,z_t)}(s_\theta(z_t,t) - \nabla_{z_t} \log q(z_t \mid z_0)) dt * \nabla_\theta s_\theta(z_t,t) = 0$$

so that

$$s_\theta(z_t,t)^* = \nabla_{z_t} \log q(z_t \mid z_0)$$

□

(B) What is the value of the loss at optimality?

Solution. It should be zero since the integrand is the squared difference between the estimated score $s_\theta(z_t,t)$ and the true score $\nabla_{z_t} \log q(z_t \mid z_0)$. If after training, these two turns perfectly match each other, the entire integral should be zero. □

Imagine

- $z_0 \sim \frac{1}{2}\text{Normal}(-10,1) + \frac{1}{2}\text{Normal}(10,1)$
- $dz_t = -\frac{1}{2} z_t dt + dW_t$

where W_t is the Brownian motion. For this process the transition $q(z_t|z_0) = \text{Normal}(\exp(-0.5t)z_0, 1 - \exp(-t))$.

(C) Plot the distribution of z_t numerically with simulation.

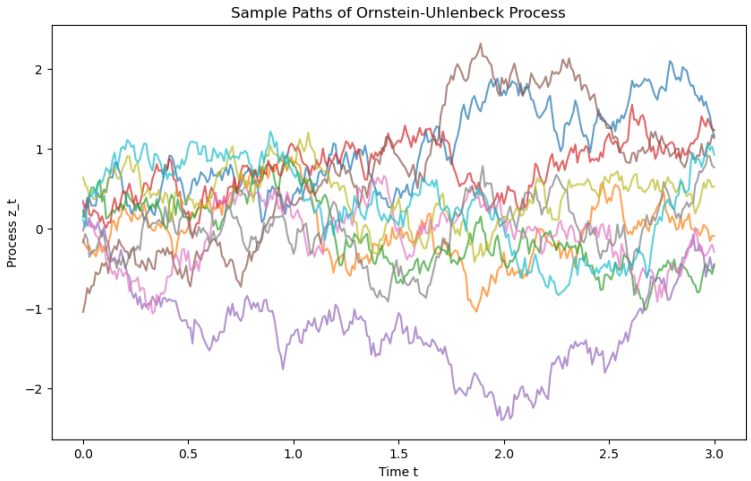


Figure 3: Output Plot

Solution. □

(D) For this example, compute the optimal $s_\theta(z_t, t)$.

Solution. We just need to compute $\nabla_{z_t} \log q(z_t \mid z_0) = \frac{z_t - \exp(-0.5t) z_0}{1 - \exp(-t)}$, which is the optimal $s_\theta(z_t, t)$. □

(E) What values of t contribute more to the overall optimal loss?

Solution. Contributions to the overall loss will depend on how this quantity changes with t. Typically, the beginning and end of the diffusion process are most informative because these are the points where the data distribution and the noise distribution are most distinct. □

(F) What does this question tell you about training a diffusion model?

Solution. We learn that certain time steps in the diffusion may be more critical to model accurately than others, and that attention should be paid to those when designing and training diffusion models. This can have implications for how we parameterize our model and how we schedule our learning rates or training regimes.

□

4 Causal Inference: Doubly Robust Estimators

Denote unit i's treatment, outcome, and its vector of covariates by T_i, Y_i, and X_i. Let us model propensity scores by $e(x;\hat{\theta})$ and the outcome by $f(x;\hat{\psi})$. Assume strong ignorability and positivity hold. We define the Doubly Robust Estimator (DRE) for the average outcome when treated, $\mathbb{E}[Y^{(1)}]$, by:

$$\frac{1}{n}\sum_{i=1}^{n}\left[\frac{T_i Y_i}{e(X_i;\hat{\theta})} - \frac{T_i - e(X_i;\hat{\theta})}{e(X_i;\hat{\theta})} f(X_i;\hat{\psi})\right] \tag{2}$$

(A) Suppose the propensity model is correctly specified, i.e. $e(x;\hat{\theta}) = P(T_i = 1|X_i = x)$. Given any function f, show that the DRE is unbiased.

Solution. □

(B) Suppose the model $f(x;\hat{\psi})$ is correctly specified, i.e. $f(x;\hat{\psi}) = \mathbb{E}[Y_i^{(1)}|X_i = x] := \mathbb{E}[Y_i|X_i = x, T = 1]$. Given any function e taking values in $(0, 1)$, show that the DRE is unbiased.

Solution. □

(C) Recall that control variates improve Monte Carlo estimators by defining a new estimator with the same expectation but lower variance. For some estimator f, this is done by taking a function g with $\mathbb{E}[g(x)] = 0$, and defining the new estimator as $\hat{f}(x) = f(x) - ag(x)$ for some $a \in \mathbb{R}$. Here $\mathbb{E}[\hat{f}] = \mathbb{E}[f]$, but the variances are not equal. The value of a that makes the variance of $\hat{f}$ smallest is $a^* = \frac{Cov(f,g)}{Var(g)}$.

When both $e(x;\hat{\theta})$ and $f(x;\hat{\psi})$ are correctly specified, use control variates to justify the use of Doubly Robust Estimators.

Solution. □

5 Effect of Confounders

In this question, you will simulate a patient database to determine whether to give Treatment A or Treatment B. For every patient i, Z_i is the level of sickness and is sampled from a Gaussian $\mathcal{N}(1,1)$. A positive Z_i indicates sickness, whereas a negative Z_i indicates health. The patient's willingness to take risks, R_i, is sampled from $\mathcal{N}(0,0.5)$. A positive R_i indicates the openness to taking risk, whereas a negative value suggests caution. The probability of giving Treatment A or Treatment B is given by

$$\mathbb{P}(T=A) = \sigma(Z+R-0.5)$$
$$\mathbb{P}(T=B) = 1-\sigma(Z+R-0.5)$$

where $\sigma(t) = 1/(1+e^{-t})$ is the sigmoidal logistic function.

Treatment A, on average, more effectively reduces sickness levels compared to Treatment B but has a small risk of causing harm. The recovery probability is given by

$$\mathbb{P}(H=1) = 1-\sigma(Z-E_A\mathbb{I}(T=A)-E_B\mathbb{I}(T=B))$$
$$\mathbb{P}(H=0) = \sigma(Z-E_A\mathbb{I}(T=A)-E_B\mathbb{I}(T=B))$$

with $E_A \sim \mathcal{N}(1.5,1)$ and $E_B = 0.5$.

(A) Draw the causal graphical model for the described patient database.

Solution. s □

(B) Simulate $N = 1000$ patients numerically. Given only the data on administered treatments and recovery outcomes, organize the findings into a 2×2 table for the combinations of $\{T=A, T=B\}\times\{H=0, H=1\}$. Estimate the Average Treatment Effect (ATE) and recommend a treatment based on the ATE. Is this an informed recommendation? Why or why not?

(C) In this part, you have also access to the level of sickness of the patients. Describe how you use this information to refine the ATE estimate and report this estimate. Does it change the your recommendation compared to part (B)? Why or why not?

6 Reinforcement Learning with Sparse Rewards

Suppose that you have a robot that should learn to move from any of a set of starting positions $s_0 \in \mathcal{S}_0$ to a goal position G. The robot can move by performing a sequence of small, low-level continuous actions, such as rotating its joints. However, you do not know how to explicitly program a sequence of actions that will move the robot from any s_0 to G. You decide to use a reinforcement learning algorithm. Your RL algorithm uses the policy gradient to learn a policy $\pi_\theta(a|s)$ that maps from states s to distributions over actions a. You consider learning episodes of a finite number of T steps.

(A) You start by designing the Markov Decision Process (MDP) that defines the robot's learning environment. The robot receives reward $R = 1000$ when it reaches location G and $R = 0$ upon entering any other position. What is the Monte Carlo estimate of the policy gradient when G is not reached in T steps in any of the sample trajectories? What happens as the robot explores in this environment and what will its learning process look like?

Solution. Based on

$$r_\tau = \sum_{t=1}^{T} \gamma_t r_t$$

$$\hat{\nabla}_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{S} \sum_{s=1}^{S} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}} (\tau_s) \, r_{\tau^s}$$

in the lecture slide, we know that if G is not reached in T steps, then r_τ would be zero, and thus sampling from the current policy would result in r_{τ^s} being zero and the MC policy gradient would be zero too so our policy won't be effectively updated.
But as the learning process goes on, G will be reached in T steps and we will sample some trajectories with non-zero rewards and the gradient would be non-zero and our parameter θ will be updated so that the policy is updated. Over time, the robot is expected to learn a more efficient policy that leads to the goal more consistently. □

(B) Suppose the robot must start in s_0 for all learning trajectories. Name two ways you could alter the MDP environment to improve exploration and gradients.

Solution. (a) We could encourage the policy to explore as more states as possible by including greater rewards $R(s, a, s')$ where $s = s'$ so that the policy will take less actions on repeated states. In the case above in (A), doing so will increase the probablity of reaching the goal state G and train the policy faster.

(b) We could use ϵ-greedy algorithm will at each time step before action, we flip a biased coin parametrized by ϵ. With probability ϵ, we act randomly(forgetting about the current policy at time step t) and with probability $1 - \epsilon$ we act on the current policy. But we should not choose ϵ to be too close to 1 since then we are not using the learning results.

□

7 Self-Attention Layer and Embeddings

You are given a large text corpus from web in the format of a sequence of discrete tokens $(x_1, ..., x_N)$. In this question, you will build a self-attention block of a large language model and explore its mechanism.

(A) Collect the input sequence in a data matrix $X = [x_1, ..., x_N] \in \mathbb{R}^{d \times N}$. Your want to map tokens from a symbolic space to a vector space using linear maps. Define the query matrix as $Q = W^Q X \in \mathbb{R}^{k \times N}$ and the key matrix as $K = W^K X \in \mathbb{R}^{k \times N}$ where W^Q, W^K are $k \times d$ matrices of learnable weights. Write out the expression for the attention weights $a_{nm} \in [0, 1]$ using Q_n (the n-th column of Q) and $K_{m'}$ (the m'-th column of K).

Solution. We know that

$$z_{nm} = Q_n^\top K_{m'}$$

before softmax layer, and after it we have

$$a_{nm} = \frac{exp(Q_n^\top K_{m'})}{\sum_{i=1}^{N} exp(Q_n^\top K_i)}$$

where N is the number of samples in the data set. □

(B) The value matrix $V = W^V X \in \mathbb{R}^{k' \times N}$ is constructed similarly, where W^V is a $k' \times d$ matrix. Write out the output of the attention module in terms of a_{nm} and V_m.

Solution. The attention output vector for a particular data point X_n is just the weighted sum of the $V_j, \forall j = 1, 2, \cdots, N$, which is the following expression:

$$b_n = \sum_{j=1}^{N} a_{nm} V_m, \quad \forall n = 1, 2, \cdots, N$$

□

(C) Consider that the query and key matrices are identical ($Q = K$). For a large embedding dimension, i.e. $k \gg d$, assume that embedding vectors are nearly orthonormal

$$\|Q_n\| = 1 \quad Q_n^T Q_{n'} = \delta$$

with some small δ. Describe the attention weights under these conditions. What would be the role of this attention module in a large network? What is missing?

Solution. If the training process leads to $Q = K$, then since a_{nm} is the inner products between nth column of Q and m'th column of K. $\|Q_n\| = 1$ would mean that $Q^\top K$ will be nearly orthogonal, which means that the each token will primarily attend to itself with $z_{nm} = \begin{cases} 1 & n = m \\ 0 & \text{otherwise} \end{cases}$ and a_{nm} biased towards the nth token input. What's missing here might be the cross-token interactions that are important for understanding contextual relationships in sequences. □

(D) Consider that $\|Q_n\| = 1$ and embeddings of similar tokens have a large dot product ($Q_n^T Q_{n'} = 1 - \epsilon$). Describe in words the corresponding attention weights. What would be the role of this attention module in a large network? What is missing?

Solution. Since a_{nm} is obtained by Q_n and $K_{m'}$, when $Q_n = Q_m$(similar embedding), then the output attention weights $a_{ik} \forall k$ and $a_{jk} \forall k$(ith token and jth token) would be similar to each other, which means that the output $b_i = \sum_m a_{im} V_m$ and $b_j = \sum_m a_{jm} V_m$ will be similar. This means that the model will pay similar attention to each of these two tokens. What might be missing is the ability to distinguish or attend to distinct but contextually relevant tokens. □